

C++面向对象程序设计期末题

缺少答案解析

按试卷分组，并为每个答案解析块补充题号

期末 2

第 1 题

答案：B

解析：私有成员和保护成员通过访问控制将数据与操作封装在类内部，对外隐藏实现细节，这正是封装的核心思想。抽象强调概念提取，继承强调层次复用，多态强调接口统一，均与访问控制无关。

第 2 题

答案：A

解析：头文件会被多个源文件包含，其中不应出现普通函数的定义（非 inline），否则链接阶段会因重复定义而报错。函数声明、类定义、外部变量声明均可安全置于头文件。

第 3 题

答案：B

解析：多文件结构的核心好处是模块化、可维护性和增量编译；它将一个程序拆分为多个翻译单元，而非用于同时生成多个独立的可执行程序。

第 4 题

答案：C

解析：const int* 是指向常量的指针，它可以指向非常量变量，只是不能通过该指针修改目标值。A 中 pd 未初始化即解引用属于未定义行为；B 数组越界访问 a[2][3]；D 中 int 引用不能绑定到 double 类型。

第 5 题

答案：D

解析：对象是类的实例，吉林大学是“学校”这一抽象概念的具体实例，最能体现对象与类的实例化关系。父亲与儿子属于继承关联，足球与运动员、书与书店属于普通关联。

第 6 题

答案：D

解析：子类完全可以定义与父类同名的成员函数，这会产生覆盖（虚函数）或隐藏（非虚函数）的效果，并非不允许同名。其余选项均符合公有继承的基本特征。

第 7 题

答案：D

解析：含虚函数的类会额外携带虚函数表指针（vptr），在 32 位环境下占 4 字节，64 位环境下占 8 字节；其余选项仅含普通数据成员，无此额外开销。因此含虚函数的类实例化后对象体积最大。

第 8 题

答案：A

解析：虚基类专门用于解决多重继承中的菱形继承问题，避免共同基类在派生类中产生多份副本及相应的名字冲突，并非为了接口补充或数据保护。

第 9 题

答案：C

解析：采用成员对象 B b 时，A 完全可以在构造函数初始化列表中显式调用 B 的带参构造函数，因此 B 并非必须拥有无参或全缺省构造函数。A、B 选项关于拷贝构造与抽象类的表述均成立。

第 10 题

答案：C

解析：类内直接初始化 `const int` 属于 C++11 支持的初始化语法，此处 0 是 `val` 的初始值，而非缺省值（缺省值特指函数参数的默认值）。其余选项对缺省值、纯虚函数、构造函数初始化列表的表述均正确。

期末 3

第 1 题

答案: D

解析: 名字空间通过将标识符封装在独立的作用域中, 避免不同模块或库之间的命名冲突。友元用于突破封装访问私有成员, 虚函数实现动态多态, 函数重载允许同名函数共存但要求参数列表不同, 三者均不直接解决命名冲突问题。

第 2 题

答案: C

解析: 头文件会被多个源文件重复包含, 其中不应放置普通函数的定义 (非 inline), 否则链接阶段会因重复定义而报错。extern 声明、const 常量定义、类定义均可安全置于头文件。

第 3 题

答案: B

解析: 函数重载要求同名函数的参数列表不同 (类型、个数或顺序)。B 中两函数参数个数不同, 构成最清晰的重载。A 仅返回值不同, 不满足重载条件; C 中 const int& 与 int 虽类型不同, 但调用时易产生二义性; D 中 int 与 intconst 为顶层 const, 不构成重载。

第 4 题

答案: B

解析: 该函数为 const 成员函数, this 指针类型为 const A, 无法隐式转换为返回类型 A; 形参 a 在函数返回后即销毁, 返回 &a 或 dynamic_cast<A*>(&a) 均会得到悬空指针。静态数据成员生命周期与程序一致, 返回其指针安全合法。

第 5 题

答案: B

解析: 多态的核心特征就是同一消息 (如函数调用) 作用于不同对象时, 根据对象实际类型产生不同行为结果。继承是类之间的关系, 动态绑定是多态的实现机制, 静态绑定则不会产生不同结果。

第 6 题

答案: A

解析：类 B 的对象仅在 A 的某个成员函数内部临时定义和使用，并非 A 的成员变量，说明 B 只是 A 在特定功能执行过程中的临时协作对象，属于典型的依赖关系。聚合与组合要求 B 作为 A 的成员存在，继承则要求类之间的层次结构。

第 7 题

答案：D

解析：当局部变量与全局变量同名并隐藏后者时，可通过作用域限定符 `::` 直接访问全局变量，如 `::var`。

第 8 题

答案：B

解析：动态联编（动态绑定）的核心特征是：在程序运行期间根据对象的实际类型来确定调用哪个虚函数。编译时确定调用目标的是静态联编。

第 9 题

答案：C

解析：类 A 以类 B 的对象作为数据成员，且 B 的生命周期随 A 的创建而创建、随 A 的销毁而销毁，这种强拥有的整体-部分关系属于组合关系。

第 10 题

答案：A

解析：若派生类继承了基类的纯虚函数却未全部实现，则该派生类仍是抽象类，尽管它自己没有“定义”新的纯虚函数。因此“没有定义纯虚函数就必定不是抽象类”的说法错误。

第 11 题

答案：错误

解析：使用未初始化的指针属于未定义行为，程序可能崩溃或产生脏数据，但编译器通常不会报语法错误。

第 12 题

答案：错误

解析：运算符重载只能改变运算符对自定义类型的操作语义，不能改变其固有的优先级和结合性。

第 13 题

答案：错误

解析：若左右两侧类型不同，赋值时会发生隐式类型转换甚至截断（如将 `int` 赋给 `char`），此时 `x` 的值并不“总是”等于右侧原始表达式的值。

第 14 题

期末 4

第 1 题

答案: B

解析: C++ 标识符的作用域分为块作用域（局部作用域）、类作用域、名字空间作用域以及文件作用域等，并不存在“对象作用域”这一分类。

第 2 题

答案: A

解析: 调用 `f(0)` 时传递的是字面常量 `0`，它无法绑定到非 `const` 引用 `int&` 上，编译会失败。若形参为值传递、指针或 `const` 指针，字面量 `0` 均可通过转换匹配。

第 3 题

答案: C

解析: A 项中形参 `y` 缺少类型声明；B 项中引用形参 `int&` 不能绑定到字面常量 `0`；D 项默认实参必须从右向左连续出现，不能跳过右侧形参而只给左侧提供默认值。C 项返回类型为 `const float`，语法正确。

第 4 题

答案: C

解析: 非静态成员函数隐含一个 `this` 指针，其类型为 `A* const`（指向 A 的常量指针）。若成员函数本身为 `const` 函数，则 `this` 类型为 `const A* const`。

第 5 题

答案: A

解析: 静态成员函数没有 `this` 指针，只能直接访问静态数据成员，不能直接访问非静态数据成员。其余选项均正确：静态数据成员可以是其所属类类型；静态成员可作为成员函数默认实参；静态成员被该类及其派生类的所有对象共享。

第 6 题

答案: B

解析: 友元函数虽然在类体内声明，但它并非该类的成员函数，而是普通函数被授予了访问私有成员的特权。其余选项均符合友元机制的规定。

AutoPlant 用外层循环模拟每一天，内层遍历 6 块地：若为空则种植（累计达 count 后跳过）；若未成熟则施肥；若成熟则收获并累加收益。每天结束调用 GrowDay()，直到全部收获后输出总天数、总成本与总收益。

解析：本题考查多态与工厂模式的综合运用。通过 Clone() 实现蔬菜的复制种植，利用虚函数 Gain() 让不同蔬菜按各自公式计算收益。AutoPlant 的关键是状态机思想：每块地每天只能处于“种植/施肥/收获”之一，且需判断累计种植数量是否已达上限。

期末 5

第 1 题

答案: C

解析: 宏定义是预处理机制，不属于面向对象程序设计的核心范畴。抽象、多态、继承均为面向对象的基本特征。

第 2 题

答案: C

解析: `const` 成员函数中，`this` 指针为 `const A*`，故不能修改本对象的 `num`；但形参 `a` 为非 `const` 引用，修改 `a.num` 合法。A 试图修改 `const` 引用 `b` 的成员，D 涉及对 `const` 对象赋值，均无法编译。B 修改本对象成员，亦非法。

第 3 题

答案: C

解析: 函数返回类型为 `A&`（非 `const` 引用），而形参 `a` 的类型为 `const A&`，`const` 属性无法通过返回 `A&` 自动剥离，因此返回 `a` 一定会导致编译错误。其余选项返回的对象生命周期均不受限于函数栈帧。

第 4 题

答案: D

解析: 抽象类虽不可直接实例化，但派生类构造时仍需调用其构造函数，且虚析构函数对资源释放至关重要；不过编译器生成的默认构造与析构在语法层面已能满足基本需求，故“没有必要自定义”在此语境下成立。A、B、C 均存在明显事实错误：抽象类完全可以包含数据成员；其基类不必是抽象类；只要存在虚函数，虚函数表就一定会被创建。

第 5 题

答案: D

解析: A、B、C 均将 `B` 作为 `A` 的内部状态（成员变量），导致类之间存在持续性的结构依赖。D 仅在函数参数中临时使用 `B`，调用结束后关系即解除，耦合程度最弱。

第 6 题

答案: 错误

解析：const 版本与非 const 版本构成合法的重载。非 const 对象优先匹配非 const 版本，const 对象只能调用 const 版本，编译器能够明确区分，不会产生二义性。

第 7 题

答案：错误

解析：虚基类用于解决菱形继承中的重复继承问题，名字空间才是专门用于解决命名冲突的机制，二者作用域不同。

第 8 题

答案：正确

解析：成员函数的代码存储在进程的代码段，由同类所有对象共享；对象内存中仅存放数据成员（及可能的虚函数表指针），并不保存成员函数的入口地址。

第 9 题

答案：正确

解析：静态数据成员属于类级别，在程序生命周期内仅初始化一次，所有对象共享同一副本，其值在多次对象创建之间保持延续。

第 10 题

答案：错误

解析：派生类完全可以定义与基类原型相同的非虚函数，此时基类函数被隐藏（hide）。只有需要动态绑定时，才必须将该函数声明为虚函数。

第 11 题

答案：

- D
- C
- B
- E
- D
- C
- B

(b(other.b))；赋值运算符需分别调用 A::operator= 和 b 的赋值操作；析构函数按逆序自动调用 B 和 A 的析构。若需指针成员，应在析构函数中手动释放，并确保拷贝时执行深拷贝。

第 27 题

答案：

解析：设计 ArtWork 为抽象基类，包含作者、标题、创作年份三个数据成员及纯虚函数 Show()。Painting 继承 ArtWork，增加宽、高属性，Show() 输出绘画尺寸；Music 继承 ArtWork，增加调性信息，Show() 输出音乐调性；Chamber 继承 Music，进一步增加演奏人数，Show() 先调用 Music::Show() 再输出人数。

Artist 类包含 Show(const ArtWork&) 成员函数，内部调用 art.Show()，利用多态统一处理不同类型的艺术品展示。

第 28 题

答案：

解析：采用双分派（访问者模式）实现：定义 Role 为访问者基类，声明 Visit(MoneyBox&) 和 Visit(PoisonBox&) 两个纯虚接口；定义 Box 为元素基类，声明纯虚函数 Accept(Role&)。MoneyBox 和 PoisonBox 在 Accept 中分别调用 role.Visit(*this)，将自身类型回传给 Role。Soldier 和 Mage 继承 Role，各自实现两个 Visit 函数，完成金钱增加、生命值扣减与金钱奖励的具体逻辑。新增角色时，只需新增 Role 派生类并实现 Visit 接口；新增宝箱时，只需新增 Box 派生类并在 Accept 中调用 role.Visit(*this)，既有的 Soldier、Mage、MoneyBox、PoisonBox 均无需修改，满足开闭原则。

期末 6

第 1 题

答案: D

解析: C 语言是面向过程的语言, 不支持面向对象特性; C++ 在 C 基础上扩展了面向对象支持。两者并非都支持面向对象。

第 2 题

答案: C

解析: string 是标准库中的类名, 并非 C++ 语言关键字。struct、template、catch 均为 ANSI C++ 关键字。

第 3 题

答案: B

解析: new A(5) 创建对象时传入参数 5, 匹配带参构造函数, 因此调用自定义有参构造函数而非无参或默认拷贝构造。

第 4 题

答案: C

解析: 静态成员函数不依附于具体对象, 编译器不会为其提供 this 指针, 因此函数体内无法使用 *this。

第 5 题

答案: C

解析: new 分配的内存需手动 delete 释放, 不会自动回收; 分配失败可能抛出异常; 若类 A 定义了析构函数, delete[] 会调用析构, 可能有输出。C 选项中变长数组 char array[n] 非标准 C++ 语法, 而 new char[n] 是合法的动态数组申请。

第 6 题

答案: C

解析: 初始化列表只能初始化直接基类和本类成员。基类的非静态成员应由基类自身构造函数初始化, 派生类无权直接初始化。

第 7 题

答案: D

解析：A、B、C 均为正确描述。基类析构函数设为 `protected` 时，派生类析构函数中仍可调用它，因此不必是 `public`；故"必须公有"的说法过于绝对。

第 8 题

答案：D

解析：构造函数、拷贝构造函数、静态成员函数均不能声明为虚函数。常成员函数（尾部加 `const`）允许设为虚函数。

第 9 题

答案：B

解析：友元机制只针对函数或类开放访问权限，不能将变量声明为友元。

第 10 题

答案：D

解析：抽象类虽不能实例化，但仍可定义构造函数完成成员初始化；且通常需要虚析构函数以确保派生类资源正确释放。因此"没有必要自定义"的说法错误。

第 11 题

答案：错误

解析：顶层 `const`（修饰指针本身）不构成重载区分条件，编译器视二者为同一函数签名，无法共存。

第 12 题

答案：错误

解析：类中允许所有数据成员均为静态，此时类相当于一个命名空间作用域，语法上完全合法。

第 13 题

答案：错误

解析：基类与派生类数据成员允许同名，派生类对象会隐藏基类同名成员，通过作用域解析符可显式访问基类版本。

第 14 题

答案：错误

解析：`protected` 继承仅在派生类内部及后续派生层次允许向上转型，在类外部将派生类对象地址赋给基类指针属于非法转换。

```

exp += e;

while (exp >= 500) { exp -= 500; level++; }
}
void reduceExp(int e) { exp = (exp > e) ? exp - e : 0; }
void levelUp() { level++; }
void CatchFish(Fish* f) { f->affect(*this); }
int getLevel() const { return level; }
int getExp() const { return exp; }
};

class Octopus : public Fish {
int w;
public:
Octopus(int weight) : w(weight) {}
void affect(Cat& c) { c.addExp(2 * w); }
};

class Whale : public Fish {
public:
void affect(Cat& c) { c.addExp(200); }
};

class Turtle : public Fish {
public:
void affect(Cat& c) { c.levelUp(); }
};

class Shark : public Fish {
int w;
public:
Shark(int weight) : w(weight) {}
void affect(Cat& c) { c.reduceExp(5 * w); }
};

```

解析：将各种鱼抽象为 Fish 基类，通过虚函数 affect 实现多态效果；Cat 类封装等级与经验规则，CatchFish 接收 Fish 指针并触发多态行为，未来新增鱼种只需继承 Fish 并重写 affect，无需修改 Cat 类代码。

期末 7

第 1 题

答案：D

解析：C++ 最核心的改进是引入面向对象机制。类和对象封装了数据与操作，支持继承、多态，这是 C 语言不具备的，因此也是最重要的改进。

第 2 题

答案：A

解析：类远比 C 的结构体强大：它支持访问控制、成员函数、继承、多态等。把类简单等同于结构体、对象等同于结构体变量，忽略了面向对象的本质特征，因此该说法错误。

第 3 题

答案：B

解析：成员函数既可以在类体内直接给出实现（隐式内联），也可以在类体外通过作用域解析符实现；类外实现时也不必与类声明放在同一文件，只需包含声明即可。

第 4 题

答案：D

解析：构造函数可以重载，但析构函数不能；构造函数的参数可以部分带缺省值；析构函数不能用 `const` 修饰。析构函数体内访问本类非静态成员完全合法，虽然技术上也能访问基类非静态成员，但通常不建议，因此 D 相对正确。

第 5 题

答案：A

解析：静态数据成员属于整个类，所有对象共享同一份存储；静态成员函数没有 `this` 指针，既可以通过对象调用，也可以通过类名调用。

第 6 题

答案：D

解析：`const` 修饰的内置类型变量必须初始化，因此 A、B、C 均因未初始化而编译错误。D 声明的是“指向常量的指针”，指针本身不是常量，允许暂不初始化，语法合法。

第 7 题

答案: B

解析: 内联函数在编译期将代码嵌入调用点, 省去函数调用开销, 从而提高执行速度。它既可以在类体内实现, 也可以在类体外通过 `inline` 关键字显式声明, 并非只能在类体内或类体外。

第 8 题

答案: C

解析: 派生类继承基类的所有成员, 包括私有成员; 只是私有成员在派生类中不可访问, 并非没有被继承。

第 9 题

答案: A

解析: 公有继承下, 派生类对象可以赋值给基类对象、初始化基类引用、把地址赋给基类指针 (发生对象切片), 但派生类对象并不能直接访问基类的私有成员。

第 10 题

答案: C

解析: 虚函数不能是静态成员函数, 必须是类的非静态成员函数; 含有虚函数的类不一定是抽象类。基类声明虚函数后, 派生类中重写该函数时不必再写 `virtual`, 编译器会自动维持其虚函数属性。

第 11 题

答案: 错误

解析: 构造函数可以重载, 但析构函数只有一个且不能重载, 因为析构函数无参数、无返回值, 签名固定。

第 12 题

答案: 错误

解析: `A b = a;` 是拷贝构造 (对象初始化), 调用的是拷贝构造函数; 赋值运算符发生在对象已创建后的 `b = a;` 场景。

第 13 题

答案: 错误

解析: 静态成员函数没有 `this` 指针, 只能访问静态成员; 常成员数据属于具体对象, 静态成员函数无法访问。

第 14 题

```
dept.SetEmployee(e3, 2);  
  
dept.ShowEveryAward();  
delete e1; delete e2; delete e3;  
return 0;  
}
```

解析：将 Employee 设计为抽象基类，提取公共接口 Award，Manager 与 Staff 分别实现各自奖金公式。

main 中按题目给定的工资与部门利润构造对象，人均奖金基数为 $300000 \times 0.3 \div 3 = 30000$ ，程序依次输出：

经理： $8000 \times 2 + 30000 \times 2 = 76000$

员工 1： $5000 + 30000 \times 0.8 = 29000$

员工 2： $4000 + 30000 \times 0.8 = 28000$

期末 8

第 1 题

答案: C

解析: 静态成员函数没有 `this` 指针, 本质上不依赖于某个具体对象, 因此不能被 `const` 修饰。通过对象调用静态函数是允许的, 但写成 `a.f() const` 这种形式属于语法错误。

第 2 题

答案: D

解析: 函数返回的是 `const A&`, 要求所引用的对象在函数结束后仍然有效。局部自动变量在栈上分配, 函数返回时即被销毁, 返回其引用会形成悬空引用。其余三种变量的生命周期均不受函数作用域限制, 可以安全返回。

第 3 题

答案: C

解析: `delete p` 只是释放指针所指向的堆内存, 并不会把指针变量 `p` 本身的值置为 0。如果想避免野指针, 需要手动置空: `p = nullptr`。

第 4 题

答案: C

解析: 构造函数、拷贝构造函数、析构函数都不能被 `const` 修饰, 因为它们在对象创建或销毁过程中需要修改对象状态。自定义的类型转换函数 (如 `operator int()`) 可以被 `const` 修饰, 表示不修改对象。

第 5 题

答案: C

解析: 在类体内直接包含本类的对象成员 `D aD`; 会导致无限递归: `D` 的对象里包含 `D` 的对象, 后者又包含 `D` 的对象..... 编译器会报错。包含本类指针 `D* pD`; 或基类/其他类对象则没有问题。

第 6 题

答案: C

解析: 基类子对象和成员子对象的构造顺序由它们在类定义中出现的先后顺序决定, 而不是由构造函数初始化列表中的书写顺序决定。在没有虚函数、不考虑内存对齐的理想情况下, 派生类对象大小确实等于各基类大小加上自身成员大小之和。

第 7 题

答案: C

解析: 在 `void f() const` 这种常量成员函数中, `this` 指针的类型被编译器隐式转换为 `const A* const`, 即指向常量对象的常量指针: 既不能修改指针本身, 也不能修改指向的对象内容。

第 8 题

答案: C

解析: 判断两个对象是否为同一个对象, 本质上是判断它们是否占据同一块内存。直接比较地址 `&a1 == &a2` 即可; 比较成员数据或构造实参都无法区分“同一个对象”和“两个恰好内容相同的对象”。

第 9 题

答案: C

解析: `~A` 是析构函数, 语法上不允许带参数, 也不能像普通函数那样被显式调用并传参。`~A(0)` 这种写法在 C++ 中不合法。

第 10 题

答案: D

解析: 纯虚函数只是强制派生类提供实现, 其返回值类型可以是任意合法类型, 不限于 `void`。含有纯虚函数的类即抽象类, 且纯虚函数一定是非静态成员函数。

第 11 题

答案: 错误

解析: 静态成员函数没有 `this` 指针, 因此无法直接调用非静态成员函数 (无论是否带 `const`), 因为非静态成员函数需要 `this` 指针来指明作用对象。它只能访问静态成员和其他静态成员函数。

第 12 题

答案: 错误

解析: 这两个函数参数个数和类型完全相同 (都是 `int`), 仅返回值不同, 不构成重载; 同时一个带缺省值、一个不带, 编译器在调用时会产生二义性, 无法通过编译。

第 13 题

答案: 错误

```
HelloCommand cmd;  
cmd.Execute(); // 输出 Hello  
return 0;  
}
```

期末 9

第 1 题

答案: D

解析: 名字空间通过将标识符封装在独立作用域中，避免不同模块间的命名冲突。虚基类解决菱形继承问题，虚函数实现动态多态，函数重载要求参数列表不同，三者均不直接用于解决命名冲突。

第 2 题

答案: C

解析: 对象内存中包含虚函数表入口地址（即 `vptr`），前提是类中至少声明了一个虚函数。静态成员不影响 `vptr` 存在；公有成员函数不必全是虚函数；析构函数也不强制为虚，只是含虚函数时建议设为虚析构。

第 3 题

答案: B

解析: 析构函数不允许重载，一个类有且只能有一个析构函数，即使类中有多个构造函数，析构函数也唯一。

第 4 题

答案: D

解析: `this` 指针记录当前对象的地址，其值本身可以作为成员函数的返回值（如返回 `this` 实现链式调用）。非常成员函数中 `this` 指向的数据可以修改；静态成员函数不依附于对象，根本没有 `this` 指针。

第 5 题

答案: A

解析: 在类定义中，`private`、`protected`、`public` 这三个访问说明符可以按需要出现任意多次，没有次数限制，用于灵活组织不同成员的访问权限。

第 6 题

答案: A

解析: 函数参数为 `A* const`，要求实参是普通的 `A` 指针。`new A` 返回的正是 `A`，可直接传入。`const A` 会丢失 `const` 属性，无法匹配；将对象直接赋给指针（`A p = a`）属于类型错误。

第 7 题

答案: C

解析：const 引用绑定现有对象时，只是为原对象创建别名，不会调用拷贝构造或赋值运算符，因此 aA 与 obj 的内存地址完全相同。若 f 不是 const 成员函数，aA.f() 将无法编译，故与 obj.f() 未必等价。

第 8 题

答案：C

解析：友元函数、友元类的成员函数、本类自身的成员函数都可以访问私有成员；但 C++ 的嵌套类（内部类）默认不具备访问外部类私有成员的特权，除非显式声明为友元。

第 9 题

答案：B

解析：100 是字面常量（右值），无法绑定到非 const 的 int& 形参。其余选项中，A 依靠缺省参数匹配调用，C、D 的 const int 和 const int& 均可接受右值 100。

第 10 题

答案：A

解析：静态成员函数没有 this 指针，无法定位到具体对象，因此不能直接访问非静态成员数据。静态成员函数可以访问静态成员；非静态成员函数既可以访问非静态成员，也可以访问静态成员。

答案：B

解析：函数返回类型为 A&（引用），return obj 直接返回形参 obj 的引用，类型完全匹配。new A() 和 new A(obj) 返回指针，&obj 返回地址，均与 A& 不匹配。

第 8 题

答案：C

解析：判断两个对象是否为同一实例，本质是比较它们在内存中的地址是否相同。数据值相等只能说明状态相同，可能是两个独立对象；构造函数实参相同也不能保证是同一对象。

第 9 题

答案：A

解析：成员访问运算符 “.” 右侧必须是标识符（成员名），不能是数字常量 100。其余三项均可通过运算符重载或逗号表达式实现。

第 10 题

答案：C

解析：函数重载要求同名函数处于同一作用域且参数列表不同。派生类中定义与基类原型完全相同的函数属于名字隐藏，而非重载。常成员函数与普通成员函数构成重载（const 修饰符参与区分）。

期末 10

第 1 题

答案: B

解析: 静态成员函数没有 `this` 指针, 无法访问非静态成员, 因此静态的 `f1` 不能调用非静态的 `f2`。其余三种情况均允许相互调用。

第 2 题

答案: D

解析: 对象内存中仅存放非静态数据成员和虚函数表指针 (`vp`)。静态成员、函数代码及函数入口地址均存储在对象之外的全局或代码段中。

第 3 题

答案: C

解析: 程序结束时操作系统会回收进程内存, 但这不属于语言层面的自动释放; 更重要的是, 不调用 `delete` 会导致析构函数不被执行, 造成资源泄漏。因此“会自动释放掉”的表述不严谨。

第 4 题

答案: D

解析: 构造函数 (含拷贝构造) 和静态成员函数均不能声明为虚函数。析构函数可以且通常应当声明为虚函数, 以确保通过基类指针释放派生类对象时能够正确调用派生类析构函数。

第 5 题

答案: B

解析: 编译器以 `.cpp` 文件 (翻译单元) 为单位进行独立编译, 头文件通过 `#include` 被展开到 `cpp` 中参与编译, 自身不会单独生成目标代码。

第 6 题

答案: D

解析: `&a` 是取地址运算, 产生一个临时的地址值 (右值), 无法放在赋值运算符左侧。其余选项在特定条件下均可作为左值: 赋值表达式返回左值引用、解引用返回所指对象、返回引用的函数调用结果也是左值。

第 7 题

答案：B

解析：自定义派生类拷贝构造函数时，只需在初始化列表中显式调用基类拷贝构造函数即可，完全不必强制自定义基类的拷贝构造函数，编译器生成的版本足以胜任。拷贝构造函数通常不能重载（传统语境下）、参数为同类引用、函数体内允许使用 `return;`，其余选项描述均成立。

第 7 题

答案：B

解析：虚继承的正确语法是在派生列表中将 `virtual` 置于继承方式之前，即 `class B : virtual public A`。A、C、D 的 `virtual` 位置均不符合语法规范。

第 8 题

答案：D

解析：`p` 的类型是 `A* const`，即指针本身为常量，但指向的对象非常量，因此通过 `p` 可以正常调用 `A` 的普通成员函数，不限于常量或静态成员函数。A 中调用 `p->f()` 要求 `A` 至少声明 `f`；B 中 `new B` 能赋给 `A*` 要求 `public` 继承；C 中 `new B` 要求 `B` 可默认构造，均正确。

第 9 题

答案：D

解析：类不能包含自身类型的非静态对象成员（无论是 `const` 还是非 `const`），否则对象大小将无限递归。自身类型的指针（A）、其他类的对象（B）、静态自身对象（C）均可在类内声明，没有问题。

第 10 题

答案：D

解析：下标运算符 `[]` 可以被重载为类的成员函数，以实现自定义类型的数组式访问。`sizeof`、域解析符 `::` 和三目条件运算符 `?:` 均属于不可重载的运算符。

期末 11A

第 1 题

答案: A

解析: `new A(5)`在堆上创建一个对象，并用 5 作为实参调用构造函数，随后将对象地址赋给 `pa`。B 误将单个对象构造理解为数组创建；C 中 `pa` 超出作用域时，指针变量本身被释放，但堆内存不会自动回收，需手动 `delete`；D 中编译器可自动生成默认构造和析构，未必存在显式定义。

第 2 题

答案: B

解析: 函数返回类型为 `A&`（非 `const` 引用），但 B 中 `a` 是函数内局部对象，函数返回后栈帧销毁，`a` 不再存在，返回其引用将产生悬空引用。静态成员、形参 `one` 和 `*this` 的生命周期均不受函数栈帧限制，可以安全返回。

第 3 题

答案: D

解析: 静态成员函数与普通函数一样支持重载，即同一个类中可以有多个参数列表不同的静态成员函数。无参构造函数和析构函数均只能有一个；原型完全相同的非静态成员函数不能并存（`const` 版本与非 `const` 版本属于不同原型，不算完全相同）。

第 4 题

答案: C

解析: A、B 仅涉及单个对象的构造或拷贝，开销相对固定。C 中 `return t` 会先构造局部对象 `t`，返回时触发拷贝构造（或移动构造），在不考虑编译器优化（如 RVO）的情况下，涉及额外的对象复制开销，执行成本最高。

第 5 题

答案: C

解析: `min` 的参数类型为 `int&`（非 `const` 引用），只能绑定左值。C 中字面常量 10 是右值，无法绑定到 `int&`，编译报错。A 利用返回引用进行链式赋值，合法；B 中 `y+=x` 的结果是左值（`y` 本身），可绑定；D 将较小值返回并赋给 `x`，亦合法。

第 6 题

```
D::f()
10
D::~D()
A::~A()
B::~B()
A::~A()
A::~A()
```

解析：构造顺序：先 A a(10)，再 new D(a) 时先构造基类 B（B 的成员 A 调用默认构造，再执行 B 的函数体），然后构造 D 的成员 A（拷贝构造），最后执行 D 的函数体。pB->f() 走虚函数动态绑定，调用 D::f()，其内先调用 B::f() 输出 B 的成员值 0，再输出 D 的成员值 10。delete 时因基类析构为虚函数，先调用 D 析构，再逆序析构成员与基类。最后 main 结束销毁栈上的 a(10)。

第 6 题

答案：（2）运行结果如下

```
B::B()
D1::D1()
B::B()
D2::D2()
B::B()
D1::D1()
3
D1::f()
B::k()
D1::g()
D2::f()
B::k()
D2::g()
D1::f()
B::k()
D1::g()
D1::~D1()
```

期末 11b

第 1 题

答案：D 的构造函数未显式调用基类 B 的构造函数，且 B 没有无参默认构造，编译器无法完成基类子对象的初始化。

解析：基类 B 只提供了 B(int,int)，派生类 D 的构造函数必须在其初始化列表中显式调用基类构造，例如 D(int anInt) : B(...), i(anInt)，否则编译报错。

第 2 题

答案：静态成员函数 g() 中使用了 this 指针。

解析：静态函数不属于某个具体对象，没有 this 指针；返回 this 属于语法错误。此外，将析构函数设为 private 会导致栈对象 A a 在作用域结束时无法被外部代码销毁，但本题最显眼的违规是 return this。

第 3 题

答案：类 A 的成员中包含了一个类型为 A 的对象 A a；，导致无限递归定义。

解析：若类体内包含本类对象（非指针/引用），对象大小将无限递归膨胀，编译器会报错。应改为 A* a；或 A& a；。

第 4 题

答案：类 Complex 未重载 << 运算符，直接用 cout << c1 无法通过编译。

解析：自定义类型想直接输出，必须提供 ostream& operator<<(ostream&, const Complex&) 的重载；否则编译器找不到匹配的输出操作。

第 5 题

答案：（1）运行结果如下

```
A::A()
A::A()
B::B()
A::A(const A&)
D::D(constA&)
B::f()
0
```

```

return result;
}
const char* getString() const { return data; }
private:
char* data;
};

```

解析：构造函数负责深拷贝字符串；拷贝构造与赋值运算符均需先分配新内存再复制，避免浅拷贝导致重复释放；`operator+` 先计算拼接后长度，利用临时缓冲区生成结果对象；`getString` 提供只读访问以满足输出需求。

第 14 题

答案：（参考实现）引入 `Marriage` 中间类，让 `Male` 和 `Female` 都持有 `Marriage*` 指针。

```

代码：
class Marriage {
public:
    Male* husband;
    Female* wife;
};
class Male {
public:
    Female* getWife() const { return pMarriage ? pMarriage->wife : 0; }
    void setMarriage(Marriage* m) { pMarriage = m; }
private:
    Marriage* pMarriage;
};
class Female {
public:
    Male* getHusband() const { return pMarriage ? pMarriage->husband : 0; }
    void setMarriage(Marriage* m) { pMarriage = m; }
private:
    Marriage* pMarriage;
};

```

解析：原先 `Male` 和 `Female` 直接互相包含指针，造成双向编译依赖。引入 `Marriage` 类后，两者都只依赖 `Marriage`，通过 `Marriage` 间接访问对方；`getWife()` 和 `getHusband()` 的原型与功能保持不变，成功解耦。

期末 12A

第 1 题

答案: D

解析: 基类的私有成员会被派生类继承，只是派生类成员函数不能直接访问；虚析构函数本身不是“被继承的成员”。

解析: C++中函数返回值的顶层 `const` 会被忽略，因此 `const int f()` 与 `int f()` 在函数原型层面等价，且二者均等价于 `int f(void)`。

第 2 题

答案: B

解析: 字面常量 100 是右值，无法绑定到非 `const` 引用 `int&`。若形参为 `const int&` 或 `const int` 则可正常接收，`int&` 会导致编译错误。

第 3 题

答案: C

解析: C++允许通过继承 `exception` 类或自定义任意类/结构体来创建新的异常类型，用户拥有完全的自定义权限。

第 4 题

答案: C

解析: 常量成员函数仅承诺不修改对象状态，对返回值类型没有任何限制，可以是 `void`、基本类型、指针、引用乃至类对象。

第 5 题

答案: B

解析: 析构函数不允许重载，一个类只能拥有一个析构函数。编译器不会为析构函数提供参数不同的多个版本。

第 6 题

答案: B

解析: 对象间通信的本质是消息传递，即一个对象调用另一个对象的公有成员函数，请求其完成特定操作。继承、封装、重载均不直接承担对象间通信职责。

第 7 题

答案：A

解析：模板参数 Q 代表类型名，不能作为变量参与算术表达式。`return Q + x` 试图将类型与对象相加，属于语法错误。其余选项在模板定义层面均合法，实例化时由具体类型决定能否通过编译。

第 8 题

答案：D

解析：转换函数（如 `operator int()`）将所属类的对象隐式转换为指定类型，无参数、无显式返回类型（由函数名隐含）。一个类可定义多个转换函数，且可被派生类继承。

第 9 题

答案：A

解析：静态成员函数没有 `this` 指针，不依附于具体对象，因此无法直接访问非静态成员数据（不知道操作哪个对象的数据）。静态成员函数访问非静态成员必须通过对象名或指针显式指定。

答案：new 自动计算类型大小、调用构造函数、返回正确类型指针，失败时抛异常；malloc 仅分配原始内存、不调用构造、返回 void*、需手动计算字节数、失败返回 NULL。

解析：两者本质区别在于 new 是类型安全的运算符，与 C++ 对象生命周期绑定；malloc 是 C 风格的内存分配函数，对对象一无所知。

第 8 题

答案：使用作用域解析符 :: 明确指定成员所属类；对于菱形继承，使用虚继承 (virtual) 消除重复基类子对象。

解析：当多个基类拥有同名成员时，派生类对象访问该名字会产生二义性，必须通过 基类名::成员名 指明；虚继承则从根本上避免重复存储和访问冲突。

第 9 题

答案：构造函数、析构函数和友元关系不能被继承。

解析：派生类会继承基类的数据成员和普通成员函数，但构造/析构函数负责特定类的对象创建与销毁，友元关系是单向特许，二者都不进入继承体系。

第 10 题

答案：静态成员函数没有 this 指针，而非静态成员必须通过 this 指针才能确定属于哪个对象。

解析：静态函数在类层面运行，不依附于具体对象实例，因此无法定位特定对象的数据成员。

第 11 题

答案：错误

解析：C++ 明确支持多重继承，一个派生类可以同时拥有多个直接基类。

第 12 题

答案：错误

解析：抽象类虽然不能被直接实例化，但其构造函数仍可在派生类构造时被调用，用于初始化抽象类自身的成员数据。

第 13 题

答案：正确

解析：模板类的派生非常灵活：派生类可以继续保留模板参数形成新的模板类，也可以全部特化后成为普通非模板类。

期末 12B

第 1 题

答案：类名、命名空间名、结构体名或枚举名（全局作用域时 `a` 为空）。

解析：`::` 是作用域解析运算符，左侧可以是类、结构体、枚举或命名空间等作用域名称，用于限定右侧名称的所属范围。

第 2 题

答案：`sizeof(A)`

解析：指针算术以所指类型的大小为步长。`p+1` 并非偏移 1 字节，而是偏移一个完整 `A` 对象所占的字节数，即 `sizeof(A)`。

第 3 题

答案：在类体外（全局作用域）进行初始化。

解析：静态成员属于整个类而非某个对象，必须在类定义之外单独分配空间并初始化，格式为：类型 类名::成员名 = 初值。

第 4 题

答案：`private` 成员仅类自身和友元可访问；`protected` 成员还允许派生类成员函数访问。

解析：`protected` 访问权限是专门为继承设计的：它在类外部与 `private` 一样不可见，但在派生类内部可见，从而支持子类复用父类实现。

第 5 题

答案：类中含有至少一个纯虚函数。

解析：一旦类声明了纯虚函数（如 `virtual void f() = 0;`），该类就成为抽象类，无法直接实例化，只能作为基类被继承。

第 6 题

答案：将函数返回类型声明为引用类型（如 `T&`）。

解析：返回引用意味着返回的是对象本身而非临时副本，因此函数调用表达式可以出现在赋值号左侧，例如 `f() = x;`。

第 7 题

解析：数组中存储的是基类指针，指向动态分配的派生类对象。拷贝构造必须深拷贝这些对象，否则新旧容器将共享同一块内存，析构时重复释放导致崩溃。通过多态的 clone 接口，每个派生类自行完成正确类型的复制。

第 27 题

答案：（参考实现）

代码：

```
class Dictionary {
public:
    Dictionary() { count = 0; }

    bool add(L* left, R* right) {
        for (int i = 0; i < count; ++i)
            if (*pairs[i].left == *left) return false;
        pairs[count].left = left;
        pairs[count].right = right;

        count++;
        return true;
    }

    R* find(L* left) const {
        for (int i = 0; i < count; ++i)
            if (*pairs[i].left == *left) return pairs[i].right;
        return 0;
    }
private:
    struct Pair { L* left; R* right; };
    Pair pairs[100];
    int count;
};
```

解析：采用平行数组存储键值对，add 时遍历检查左元是否已存在（保证唯一性），find 时按内容匹配返回对应右元指针。main 中创建 L、R 对象，调用 add 插入字典，再用 find 检索即可。

模拟 1

第 1 题

答案: B

解析: C++ 标准作用域包括块（局部）、类、名字空间（全局）三种，不存在所谓“对象作用域”。对象只有生命周期，没有作用域概念。

第 2 题

答案: A

解析: 字面常量 0 是右值，无法绑定到非 `const` 引用 `int&`。其余选项中，指针形参可接收 0 作为空指针；若类 A 存在单参构造函数，0 还可触发隐式类型转换。

第 3 题

答案: C

解析: A 选项 `y` 缺少类型声明；B 选项引用缺省值不能绑定到字面常量 0；D 选项默认参数必须从右向左连续出现，`int y` 无默认值而 `int x=10` 有默认值，顺序违规。C 选项语法完全合法。

第 4 题

答案: C

解析: 非静态成员函数隐含 `this` 指针，其类型为 `A* const`（指向类 A 对象的指针常量，指针本身不可修改）。若成员函数尾部带 `const` 修饰，则类型变为 `const A* const`。

第 5 题

答案: A

解析: 静态成员函数没有 `this` 指针，不依附于具体对象，因此只能直接访问静态数据成员，无法直接访问非静态数据成员（需通过对象名显式指定）。

第 6 题

答案: B

解析: 友元函数虽在类体内声明，但并非该类的成员函数，它属于全局或其他类的作用域，只是被授予了访问私有成员的特权。

第 7 题

答案：D

解析：当局部变量隐藏同名全局变量时，可通过全局作用域限定符 `::` 访问全局版本，如 `::var`。

第 8 题

答案：B

解析：动态联编（运行时多态）的核心特征是运行时根据对象实际类型确定调用哪个函数，而非编译时确定。虚机制是实现动态联编的基础，且必须通过指针或引用调用虚函数才能触发。

第 9 题

答案：C

解析：类 A 以对象成员形式包含类 B，B 作为 A 的组成部分与 A 同生同灭，属于强拥有的组合关系。聚合关系通常表现为指针或引用，生命周期相互独立。

第 10 题

答案：D

解析：虚函数表（vtable）存储的是虚函数入口地址（函数指针），函数体代码存放在代码段中，并不在虚函数表内。抽象类完全可以拥有数据成员和普通成员函数。

第 11 题

答案：错误

解析：使用未初始化指针能通过编译，属于运行时未定义行为（可能崩溃或产生脏数据），并非语法错误。

第 12 题

答案：错误

解析：重载运算符时，其优先级、结合性和操作数个数均保持原有规定，无法通过重载改变。

第 13 题

答案：错误

解析：若 x 与表达式类型不同，赋值时会发生隐式类型转换或截断。例如 `int x = 3.9`，表达式值为 3.9，而 x 实际存储为 3，二者并不相等。

第 14 题

答案：错误

解析：类定义只是类型声明，不分配内存空间。只有实例化对象（如 `A a;`）或定义静态成员时，才会为数据成员分配实际存储。

第 15 题

答案：错误

解析：除本类成员函数外，被声明为友元的全局函数、其他类的成员函数或友元类，均可直接访问私有成员。

第 16 题

答案：正确

解析：`const` 成员函数本质是对 `this` 指针加 `const` 限定，静态成员函数没有 `this` 指针，故不能声明为 `const`；构造函数负责初始化对象，析构函数负责销毁，二者均不能是 `const` 成员函数。

第 17 题

答案：错误

解析：派生类对象可以安全看作基类对象（向上转型），但基类对象不能看作派生类对象。基类缺少派生类特有的成员，强制向下转型会导致对象切片或未定义行为。

第 18 题

答案：错误

解析：继承的主要目的是代码复用和功能扩展。`protected` 成员对派生类开放，实际上在一定程度上削弱了封装性，而非改进数据隐藏。

第 19 题

答案：错误

解析：单参构造函数实现的是从其他类型（包括内置类型）到自定义类型的转换。若要将自定义类型转为内置类型，需定义类型转换函数（如 `operator int()`）。

第 20 题

答案：错误

解析：抽象类只需至少包含一个纯虚函数即可，其余成员可以是普通成员函数、数据成员、静态成员等。所有函数都是纯虚函数的类通常称为接口类，也是一种合理设计。

模拟 2

第 1 题

答案: C

解析: 抽象的本质就是提取事物的核心特征，忽略与当前问题无关的次要细节。封装强调隐藏内部实现，隐藏并非标准术语，覆盖则指子类重写父类方法，均不符合题意。

第 2 题

答案: B

解析: 对象是类的具体实例。林肯是“总统”这一抽象角色或概念的具体对象，最能体现实例化关系。其余选项中，书和图书馆属于聚合，母亲和女儿属于关联，学生和研究生属于继承。

第 3 题

答案: C

解析: 多态的核心含义是同一接口在不同对象上呈现不同行为。复用强调代码重用，继承强调层次关系，名字空间解决命名冲突，三者均不涉及“同一操作、不同实现”。

第 4 题

答案: C

解析: 公有继承下，派生类可以访问直接基类和间接基类中的公有和保护成员，但任何情况下都无法访问私有成员。因此 C 的成员函数能够访问 B 和 A 中的保护成员。

第 5 题

答案: D

解析: 若类仅包含普通对象数组，且数组元素类型本身不涉及动态资源，编译器自动生成的逐成员拷贝通常足够安全。A 涉及堆内存必须深拷贝；B 因编号唯一性需特殊处理；C 作为单例必须禁止拷贝，均需显式干预。

第 6 题

答案: C

解析: 静态成员函数不依附于具体对象，没有 this 指针，因此 this 在静态函数中根本不存在，属于编译期错误。其余选项在语法层面均可作为返回值，尽管返回局部变量引用 (D) 会导致悬空，但 this 是根本无法使用。

第 7 题

答案：B

解析：非静态数据成员随对象存储：局部对象在栈区，new 对象在堆区，全局或静态对象在全局数据区。静态数据成员不属于对象存储；成员函数地址统一存放在代码段，对象中并不保存。

第 8 题

答案：B

解析：运行时多态通过虚函数与虚函数表实现：基类指针或引用调用虚函数时，程序在运行期根据对象实际类型查表定位具体实现。静态联编在编译期确定，内联函数和运算符重载均不直接提供运行时多态能力。

第 9 题

答案：A

解析：A 仅在成员函数的参数中临时使用 B，二者之间是临时性的使用关系，属于依赖。聚合要求 B 作为 A 的成员长期存在，组合是更强的聚合，泛化即继承，均不符合。

第 10 题

答案：B

解析：抽象类只要包含至少一个纯虚函数即可，其余成员完全可以是普通函数或数据成员，因此可以有数据成员。抽象类可以拥有构造函数；每个类（包括抽象类及其派生类）各自维护独立的虚函数表。

第 11 题

答案：正确

解析：非静态成员函数中 this 是指向当前对象的指针，*this 解引用后即为目标本身，通过对象访问成员变量 x 完全合法，等价于直接写 x。

第 12 题

答案：正确

解析：自赋值（如 `a = a`）若不提前判断，后续释放原有资源或重新分配时，可能因源与目标为同一对象而导致数据丢失或内存错误，因此自赋值检查是赋值运算符的必要步骤。

第 13 题

答案：正确

答案：错误

解析：成员的初始化顺序严格由其在类定义中的声明顺序决定，与初始化列表中书写的先后顺序无关。程序员若误以为可以按列表顺序调整，反而可能因依赖顺序错误导致未定义行为。

模拟 3

第 1 题

答案: D

解析: 名字空间通过将标识符封装在独立作用域中, 避免不同模块间的命名冲突。友元用于突破访问限制, 虚函数实现动态多态, 函数重载允许同名函数共存但要求参数列表不同, 三者均不直接解决命名冲突。

第 2 题

答案: C

解析: 头文件会被多个源文件重复包含, 其中不应放置普通函数的定义 (非 inline), 否则链接阶段会因重复定义而报错。extern 声明、const 常量定义、类定义均可安全置于头文件。

第 3 题

答案: A

解析: 函数重载要求同名函数的参数列表不同, 返回值不同不构成重载条件。B、C、D 的参数在个数或类型上均存在差异, 属于合法重载。

第 4 题

答案: B

解析: 函数返回类型为 A, 且 f 为 const 成员函数, this 指针类型为 const A, 无法隐式转换为 A*; 形参 a 在函数返回后即销毁, 返回&a 或 dynamic_cast 结果均会得到悬空指针。静态数据成员生命周期与程序一致, 返回其指针安全合法。

第 5 题

答案: B

解析: 多态的核心特征就是同一消息 (如函数调用) 作用于不同对象时, 根据对象实际类型产生不同行为结果。继承是类之间的关系, 动态绑定是多态的实现机制, 静态绑定则不会产生不同结果。

第 6 题

答案: A

解析: 类 B 的对象仅在 A 的某个成员函数内部临时定义和使用, 并非 A 的成员变量, 说明 B 只是 A 在特定功能执行过程中的临时协作对象, 属于典型的依赖关系。聚合与组合要求 B 作为 A 的成员存在, 继承则要求类之间的层次结构。

第 7 题

答案：错误

解析：float 与 double 之间不存在隐式类型转换，将 double 变量的地址赋给 float 指针属于类型不匹配，编译会直接报错。

第 8 题

答案：正确

解析：const 对象只能调用 const 成员函数，若 a.fun()编译通过，则 fun 必定被 const 修饰。

第 9 题

答案：错误

解析：派生类可能未增加任何数据成员（如空派生类），此时 sizeof(B)可能等于 sizeof(A)；若编译器进行空基类优化，甚至可能出现 sizeof(B)小于 sizeof(A)的情况。

第 10 题

答案：错误

解析：单参构造函数若被 explicit 修饰，则禁止用于隐式类型转换。因此并非所有单参构造函数都能自动执行隐式转换。

第 11 题

答案：错误

解析：若类继承自含有虚函数的基类，即便自身没有显式声明虚函数，编译器仍会为该类生成虚表（用于存放继承而来的虚函数指针）。

第 12 题

答案：错误

解析：异常遵循栈展开机制，可以在当前函数中捕获，也可以抛出后由外层调用栈中的 catch 块捕获，并非只能在本函数内处理。

第 13 题

答案：错误

解析：抽象类的构造函数和拷贝构造函数可以是 public 的。虽然抽象类不能直接实例化，但派生类构造时需要通过初始化列表调用基类构造，public 访问级别完全满足此需求，不必强制设为 protected。

第 14 题

答案：错误

解析：非虚函数完全可以调用虚函数，无论是通过对象直接调用还是通过 `this` 指针调用，均不受限制。

第 15 题

答案：错误

解析：析构函数不允许重载，一个类有且只能有一个析构函数，无论构造函数有多少个重载版本。

第 16 题

答案：错误

解析：类 A 及其祖先类中均不含指针或引用型成员，说明不存在动态分配的堆内存。默认拷贝构造函数执行逐成员复制，不会涉及资源所有权转移或重复释放问题，因此不会造成内存泄漏。

模拟 4

第 1 题

答案：B

解析：将数据成员设为私有或保护，限制外部直接访问，只能通过公有接口操作，这正是封装的本质：隐藏内部实现，暴露可控行为。

第 2 题

答案：A

解析：头文件会被多个源文件包含，若其中出现函数定义（带函数体），链接时会产生重复定义错误。函数声明、类定义、extern 变量声明都可以放在头文件中。

第 3 题

答案：B

解析：多文件结构的好处是模块化、可维护、增量编译（只改动的文件重新编译），但它与“同时生成多个可执行程序”无关，后者取决于编译配置和工程结构。

第 4 题

答案：C

解析：A 中 pd 未初始化，解引用 pi 属于野指针操作；B 中 a[2][3] 越界（有效下标最大为 a[1][2]）；D 中 int 引用不能绑定到 double 变量（类型不匹配）。C 中 const int 指向普通 int 变量完全合法。

第 5 题

答案：D

解析：类是抽象模板，对象是类的具体实例。“学校”是抽象概念（类），“吉林大学”是具体存在（对象）。其余选项描述的是其他关系（继承、关联、聚合）。

第 6 题

答案：D

解析：子类完全可以定义与父类同名的函数，这会产生隐藏（hide）或重写（override）的效果，并非不允许同名。

第 7 题

答案：D

解析：A、B、C 均只含一个数据成员，对象大小基本等于一个整型。D 含有虚函数，编译器会为对象添加虚表指针（vptr），通常占 4 或 8 字节，再加上 unsigned int，因此 D 的对象最大。

第 8 题

答案：A

解析：虚基类主要解决多重继承中的菱形继承问题：当多条继承路径汇聚到同一基类时，虚继承确保派生类中只有一份共享的基类子对象，避免数据冗余和访问冲突。

第 9 题

答案：C

解析：若 A 的构造函数在初始化列表中显式调用 B 的带参构造，B 完全可以没有默认构造函数。因此“B 必须有无参或全缺省构造函数”的说法过于绝对，是错误的。

第 10 题

答案：C

解析：非 const static 数据成员不能在类体内直接初始化（const static int 等少数情况除外），必须在类外单独定义并初始化。其余三项语法均合法。

第 11 题

答案：错误

解析：const 成员函数中 this 的完整类型是 const A* const（指向常量对象的常量指针），而不仅仅是 const A*。指针本身的指向也不可修改。

第 12 题

答案：正确

解析：友元关系可以授予全局函数，也可以授予其他类的某个成员函数（如 friend void B::func();），还可以授予整个类。

第 13 题

答案：错误

解析：除了公有继承的向上转型，还可能通过 A 的转换构造函数（接受 B 类型参数）或 B 的类型转换运算符实现，因此 B 不一定是 A 的派生类。